

Automatic differentiation

A loss is a chain of simple operations. Record them on the way forward, then run the chain rule backwards to get every gradient at once.

THE QUESTION

How does a program get the exact gradient of a loss with respect to every parameter, without deriving it by hand or nudging each parameter in turn?

THE INTUITION

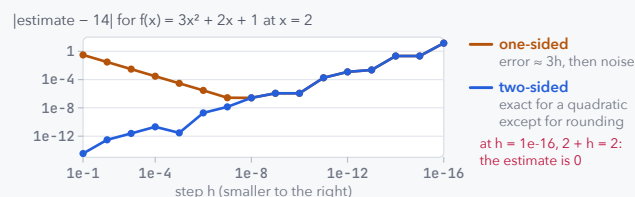
Code builds a loss from a few primitive operations with known derivatives. Record them as a graph on the forward pass, then walk it back from the loss: each node multiplies the gradient from above by its local derivative, and gradients meeting at a node add.

KEY TAKEAWAY

Reverse mode gives $\partial L / \partial \theta$ for all n parameters for about 4 forward passes; finite differences need $2n$. Gradients accumulate, so zero them before each backward pass.

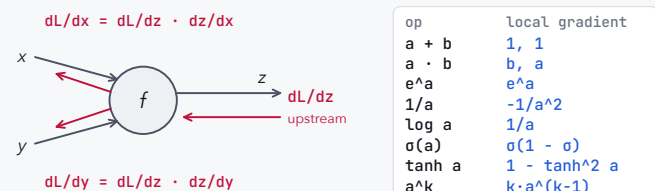
primitive ops → forward: store values → seed $dL/dL = 1$ → upstream $\times$ local, in reverse → add over paths → read θ .grad

1 Finite differences are slow and fragile



Autodiff is not numerical differentiation. The one-sided difference is off by about $hf''/2$; the two-sided $(f(x+h) - f(x-h))/2h$ cancels that term. Both cost extra passes for every input, and a tiny h subtracts nearly equal numbers: $f'(2) = 14$ comes out as 14.21 at $h = 10^{-14}$. Keep them for checking gradients.

2 Downstream gradient = upstream $\times$ local



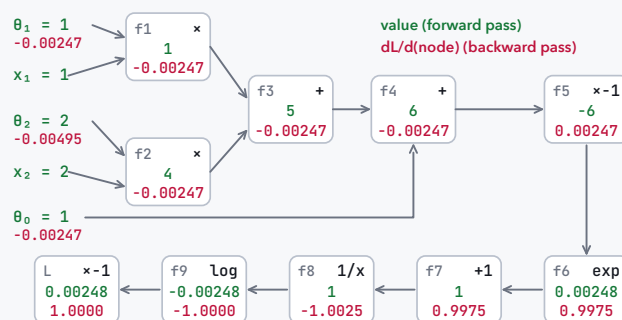
Nodes are operations and edges carry values: $(xy) + z$ is a $\times$ node feeding a $+$ node. A node $z = f(x, y)$ receives the upstream gradient $\partial L / \partial z$ and passes $\partial L / \partial z \cdot \partial z / \partial x$ to each input. A library needs only the local gradient of each primitive.

3 A bigger node gives the same answer



One sigmoid node with local gradient $\sigma(1 - \sigma)$ replaces $\times (-1)$, $\exp$, $+1$ and $1/x$ in panel 4 and gives the same $\partial L / \partial z = \sigma(z) - 1$ (the notes' exercise). Frameworks go further: a linear layer outputs logits and a cross-entropy node takes them.

4 The logistic loss, node by node



$L = -\log \sigma(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$ with $y = 1$; the backward pass starts at L with 1

The notes' example: $y = 1$, $x = (1, 2)$ and $\theta = (1, 1, 2)$, so $z = 6$ and $L = \log(1 + e^{-6}) = 0.00248$. Green values come from the forward pass. Red gradients come from the backward pass, which starts at $\partial L / \partial L = 1$ and visits the nodes in reverse order. PyTorch prints the same numbers in the autodiff notebook.

5 One path is a product of local gradients

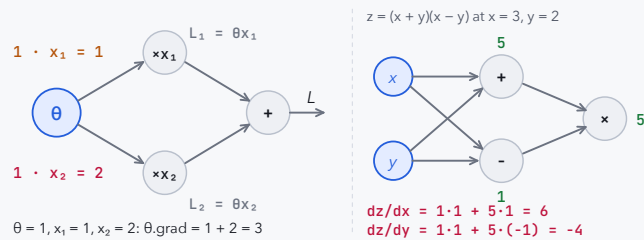
edge	local rule	local value	running product
dL/df_9	$\times -1$	-1.00000	-1.00000
df_9/df_8	$1/f_8$	1.00248	-1.002479
df_8/df_7	$-1/f_7^2$	-0.99506	0.997527
df_7/df_6	$+1$	1.00000	0.997527
df_6/df_5	e^{f_5}	0.00248	0.002473
df_5/df_4	$\times -1$	-1.00000	-0.002473
df_4/df_3	$+\theta_0$	1.00000	-0.002473
df_3/df_2	$+f_1$	1.00000	-0.002473
$df_2/d\theta_2$	x_2	2.00000	-0.004945
$dL/d\theta_2 = \text{product of the 9 local gradients}$			-0.00495

θ_2 reaches L along one path, so $\partial L / \partial \theta_2$ is the product of the nine local gradients on it; each running product is the upstream gradient at the next node down. The $1/x$ node's local gradient $-1/f_7^2 = -0.99506$ flips the sign, and e^{f_5} shrinks it 400-fold. The result equals $(\sigma(z) - y)x_2$ from the logistic regression sheet, and central differences agree to 10^{-10} .

Gradients add where paths meet

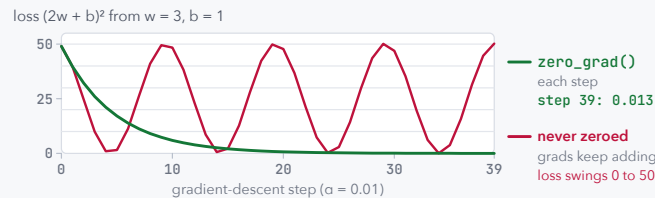
A reused value collects gradient from every path. Reverse mode suits one loss and many parameters; forward mode is its mirror image.

6 A reused value adds its gradients



A value used twice gets a gradient along each path and adds them ($dz/dt = \partial z/\partial x \cdot dx/dt + \partial z/\partial y \cdot dy/dt$), so `_back` uses `+=`. N training examples share θ : $L = L_1 + \dots + L_N$ and θ collects the sum.

7 Gradients pile up until you zero them



`.grad` is added to: three passes of $\theta x_1 + \theta x_2$ without zeroing give 3, 6, 9. Stale gradients sum all past steps, so on the notebook's $(2w + b)^2$ the loss swings between 0 and 50. Each step is:

```
opt.zero_grad()      # clear old .grad
loss = loss_fn(model(X), y)
loss.backward()      # fill .grad
opt.step()           # theta -= lr * grad
```

Evaluate under `torch.no_grad()`: no graph.

8 A scalar Value class does all of this

```
class Value: # fields: data, grad, prev
    def __mul__(self, o):
        out = Value(self.data * o.data,
                     prev=(self, o))
    def _back(): # upstream x local
        self.grad += o.data * out.grad
        o.grad += self.data * out.grad
        out._back = _back
    def backward(self):
        order = topo(self) # leaves first
        self.grad = 1.0
        for v in reversed(order): v._back()
```

`topo` is a depth-first search listing each node after its inputs. Add, exp, log and tanh work the same way; the notebook's test expression gives $\partial v/\partial x = 24.7180$, as do central differences.

9 Forward mode needs a sweep per input

seed / θ	z	σ	L	sweep
forward mode: (value, derivative) pairs, left to right				
seed θ_0	1	0.00247	-0.00247	1
seed θ_1	1	0.00247	-0.00247	2
seed θ_2	2	0.00493	-0.00495	3

reverse mode: $dL/d(\text{node})$, right to left

θ_0, θ_1 : -0.00247	-0.00247	-1.00248	1 (seed)	1
θ_2 : -0.00495				

Same gradients: forward needs one sweep per input, reverse one per output.

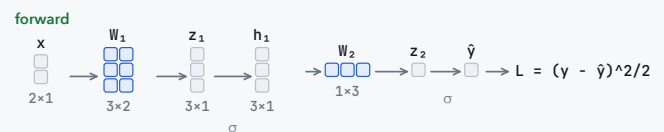
Dual numbers $a + a\varepsilon$ with $\varepsilon^2 = 0$ carry a derivative along: $(a + a\varepsilon)(b + b\varepsilon) = ab + (ab + ab)\varepsilon$. A forward sweep seeded on one parameter gives its gradient; reverse mode gets all three in one sweep. For $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$ a forward sweep gives one Jacobian column, a reverse sweep one row (see the maths for ML sheet).

10 Reverse mode costs 4 passes for any n



Flops counted in our two classes on a logistic loss with n parameters. Reverse mode costs 4 forward passes for any n but stores every intermediate value; forward mode costs about $3n$ and central differences $2n$. A network has one scalar loss and millions of parameters, so it trains with reverse mode: backpropagation.

11 Two layers in matrix form



backward ($\delta = dL/dz$, same shape as z)

$$\delta_2 = (\hat{y} - y) \sigma'(z_2) \quad dW_2 = \delta_2 h_1^T (1 \times 3)$$

$$\delta_1 = W_2^T \delta_2 \sigma'(z_1) \quad dW_1 = \delta_1 x^T (3 \times 2)$$

$db = \delta$; * is elementwise; each dW has W 's shape
 random 2-3-1 net (seed 0): matches finite differences to 5e-11

With $z = Wh + b$ at each layer and $\delta = \partial L/\partial z$, the scalar rule applied entry by entry gives these matrix products; each gradient has its parameter's shape. The multi-layer perceptrons sheet sets up the forward pass.

store values forward · upstream × local backward · add over paths · zero `.grad` · reverse mode for one loss

Before: the multi-layer perceptrons sheet; next: the next-character prediction sheet. Source: handwritten notes "Automatic differentiation", the autodiff and autograd-from-scratch notebooks, the neural networks tutorial (nipunbatra.github.io/ml-teaching). Numbers recomputed with a Value class and checked by central differences; flop counts and the 2-3-1 network are ours. Corrected from the notes: $f_0 = -0.00248$ (written -0.02); f_1 takes θ_1 (drawn as θ_0).